

Long Context

Stretching RoPE to read past the training length · video 2:56

This lesson answers one question: how can a model read texts longer than the ones it was trained on? A tiny RoPE model trained on 64 characters is asked to read 256. Used as is, it falls apart past about position 96. Three fixes are tested: **position interpolation** (squeeze the positions), **NTK-aware scaling** (raise RoPE's base) and **interpolation plus a short fine-tune**. Each fix changes one number in the RoPE code; real models such as Qwen2.5 use refined versions of the same ideas.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 7).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. The experiment	▶ 0:08-0:46	2	☐
2. No change, then squeezing	▶ 0:46-1:24	3	☐
3. Raising the base, and a short fine-tune	▶ 1:24-2:03	3	☐
4. At scale, the costs, and the code	▶ 2:03-2:47	3	☐

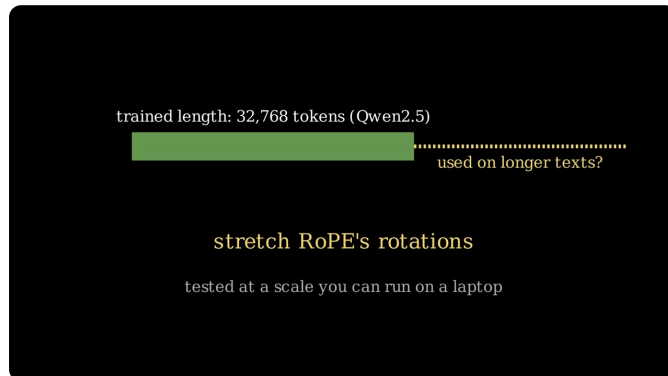
Before you start: Deep Dive episode 5 (RoPE) and LLMs in Practice episode 2 (context window, KV cache). Code: `code/d06_long_context` (trains in a few minutes on a laptop CPU, or loads the included `rope_gpt.pt`).

CONCEPT 1 OF 4

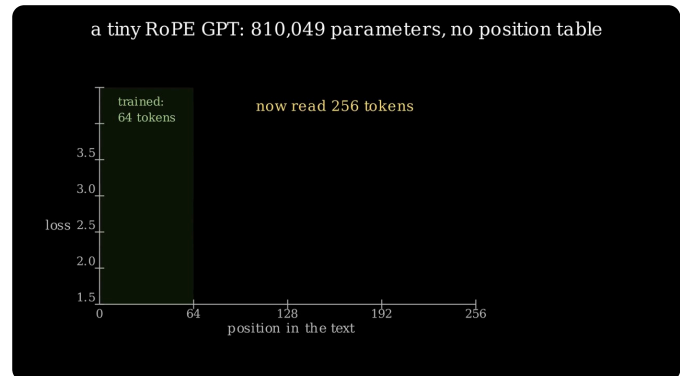
The experiment

▶ Video 0:08–0:46

Every model is trained up to some length; for Qwen2.5 that is **32,768** tokens. Yet models are used on far longer texts. With RoPE, this is done by stretching the rotations. To see it at laptop scale, the episode trains a tiny GPT on Shakespeare: **810,049** parameters, RoPE and no position table, trained on only **64** characters. Then it reads **256**, four times longer, and the loss is measured at every position.



▶ 0:27 Models are trained up to a fixed length (32,768 tokens for Qwen2.5) but used on longer texts.



▶ 0:45 The test: a RoPE model trained on 64 characters reads 256, and its loss is measured at every position.

KEY IDEA

Loss by position shows exactly where a model stops understanding a long text: low and flat is good, a climb means trouble.

Check 1: The experiment

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

The model is trained on 64 characters and tested on 256. By what factor is the test longer?

1.2 TRUE OR FALSE

“The tiny model has a learned table with one vector per position.”

True ☐ False ☐

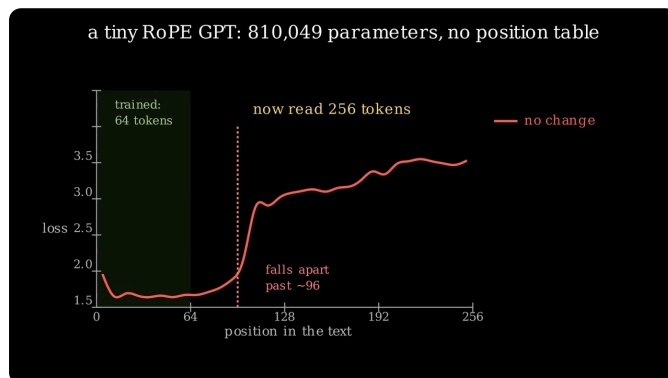
CONCEPT 2 OF 4

No change, then squeezing

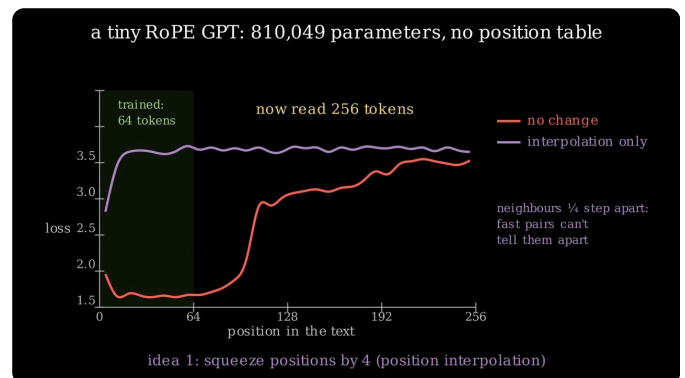
▶ Video 0:46–1:24

No change. In the first 64 positions the loss is **1.60**; from 64 to 127 it averages **2.15**; from 128 to 255, **3.23**. Past about position 96, the model falls apart: those rotation angles never appeared in training.

Position interpolation multiplies every position by $64/256 = \frac{1}{4}$, so the 256 positions are squeezed into the trained range. Without retraining, the result is bad: about **3.4** everywhere (3.43, 3.58, 3.59), even in the first 64 positions. Neighbouring tokens are now only a quarter of a step apart, and the fast pairs can no longer tell them apart.



▶ 1:01 No change: loss 1.60 on the trained positions, then a climb to about 3.4.



▶ 1:23 Position interpolation without training: about 3.4–3.6 everywhere, even in the first 64 positions.

KEY IDEA

Extrapolating shows the model angles it has never seen; interpolating keeps the angles familiar but crowds neighbouring tokens together.

Check 2: No change, then squeezing

Answer all 3 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

With position interpolation at scale $\frac{1}{4}$, which position's angles does token 200 get?

2.2 MULTIPLE CHOICE

Why does interpolation without training hurt even the first 64 positions?

- A. The model's weights change
- B. Neighbouring tokens become $\frac{1}{4}$ step apart, which the fast pairs never saw
- C. The text is longer
- D. The vocabulary changes

2.3 HANDS-ON CODE

Try it yourself. Load the trained model from `long_context.py` and try a milder squeeze: scale $\frac{1}{2}$ on 128 characters. Does halving the squeeze save interpolation without training?

```
m = copy.deepcopy(base)
m.rope = Rope(scale=0.5)
lp = loss_by_position(m, T=128)
print(lp[:64].mean(), lp[64:].mean())
```

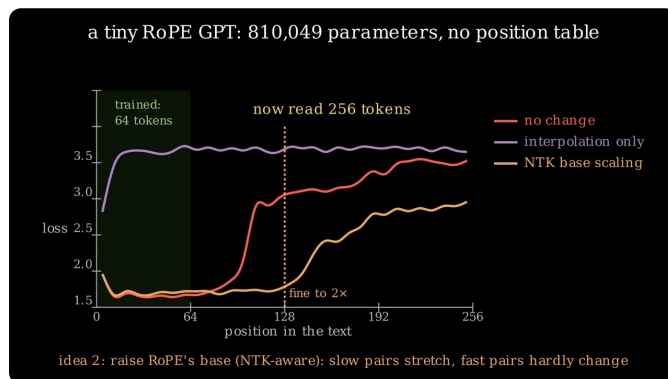
CONCEPT 3 OF 4

Raising the base, and a short fine-tune

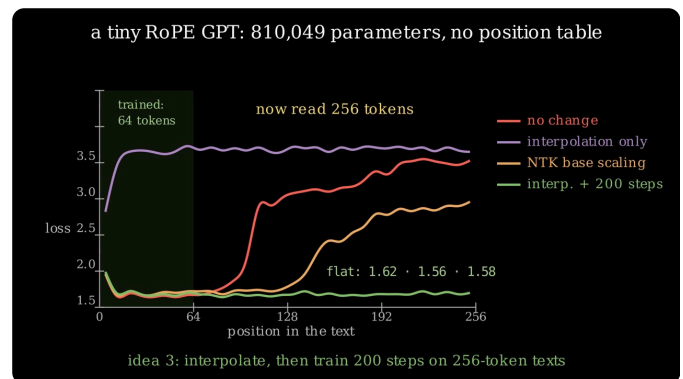
▶ Video 1:24-2:03

NTK-aware scaling raises RoPE's base instead, from 10,000 to about **43,873** ($10,000 \times 4^{32/30}$ for head size 32). The slowest pair now turns exactly 4× slower, pair 1 only 1.10× slower, and the fastest pair not at all. With no training, the loss stays near **1.6** up to position 127, twice the training length (1.63, 1.62), and only then rises (2.51 for 128-255).

Interpolation, then a short fine-tune: just 200 steps on 256-character texts. Now the loss is flat across the whole text: **1.62, 1.56, 1.58**. The model quickly learns the finer grid of angles; longer texts even help a little, because each prediction has more context.



▶ 1:43 NTK-aware scaling, no training: about 1.6 up to position 127, then a rise.



▶ 2:02 Interpolation + 200 training steps at 256: flat at 1.62, 1.56, 1.58.

KEY IDEA

Change the slow pairs, keep the fast ones (NTK), or squeeze everything and briefly retrain (interpolation + fine-tune): both beat doing nothing.

Check 3: Raising the base, and a short fine-tune

Answer all 3 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

In the code, how much slower does the slowest RoPE pair turn after NTK scaling for a factor of 4?

3.2 TRUE OR FALSE

“NTK-aware scaling slows every pair down by 4×.”

True ☐ False ☐

3.3 HANDS-ON CODE

Try it yourself. Use NTK scaling for a factor of 2 and read 128 characters. What is the loss on positions 64-127, compared with 2.17 with no change?

```
m = copy.deepcopy(base)
m.rope = Rope(base=10_000 * 2 ** (HD / (HD - 2)))
lp = loss_by_position(m, T=128)
print(lp[:64].mean(), lp[64:].mean())
```

CONCEPT 4 OF 4

At scale, the costs, and the code

▶ Video 2:03–2:47

Real models use the same recipe at scale. Qwen2.5's documentation describes **YaRN**, a refined version of NTK scaling, to stretch **32,768** tokens to **131,072**. Other model families use their own variants, usually with some long-text training.

A longer window isn't free: the **KV cache** grows with every token, and models still tend to use the **middle** of a long text less well than its start and end. Stretching the window is one thing; using it well is another.

the same recipe, at scale

trained: 32,768

YaRN-stretched: 131,072 (Qwen2.5 documentation)

other families: their own variants, usually with some long-text training

▶ 2:21 Qwen2.5's documentation describes YaRN to stretch 32,768 tokens to 131,072.

```
1 class Rope:
2     def __init__(self, base=10_000.0, scale=1.0):
3         self.freqs = base ** (-torch.arange(0, d, 2) / d)
4         self.scale = scale
5
6     def call(self, x):
7         angle = (positions * self.scale)[::None] * self.freqs # ... then rotate
8
9     pi = Rope(scale=64 / 256) # position interpolation
10    ntk = Rope(base=10_000 * (256 / 64) ** (d / (d - 2))) # NTK-aware scaling
```

▶ 2:46 Each method is one number: scale multiplies the positions, base sets the rotation speeds.

In code, each method is one number: position interpolation passes `scale` to multiply the positions; NTK scaling raises `base`. Everything else in the model stays the same.

KEY IDEA

Because RoPE is a formula, not a learned table, a model's context can be extended after training by changing its scale or base, plus a little long-text training.

Check 4: At scale, the costs, and the code

Answer all 3 questions before looking at the answers. Pass mark: all correct.

4.1 CALCULATION

By what factor does YaRN stretch Qwen2.5's context, according to its documentation?

4.2 SHORT ANSWER

A model's window is stretched from 32,768 to 131,072 tokens. Name two costs or limits that remain.

4.3 PUT IN ORDER

Order from worst to best average loss on positions 128–255: *NTK-aware scaling · interpolation + 200 steps · no change · interpolation only*.

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. The experiment · rewatch ▶ 0:08-0:46

1.1 4. $256 / 64 = 4$; every method below is tuned for this factor.

1.2 False. It has no position table at all: position comes only from RoPE rotating the queries and keys.

2. No change, then squeezing · rewatch ▶ 0:46-1:24

2.1 50. $200 \times \frac{1}{4} = 50$, inside the trained range 0-63.

2.2 B. Every position is scaled, short texts included; the model was trained on steps of 1.

2.3 No: 2.66 and 2.71, still far worse than the untouched model's 1.60 on the first 64. Even a half-step spacing is unfamiliar to the fast pairs; interpolation needs some training.

3. Raising the base, and a short fine-tune · rewatch ▶ 1:24-2:03

3.1 $4\times$ (exactly). Its frequency drops from 0.000178 to 0.0000445 rad per token; the exponent $HD/(HD - 2)$ is chosen so the slowest pair stretches by the full factor.

3.2 False. The fastest pair is unchanged (1.0 rad per token) and pair 1 slows by only $1.10\times$; only the slowest pairs stretch by the full factor.

3.3 1.61 and 1.73. Raising the base keeps the first 64 positions intact and makes the second half usable without any training.

4. At scale, the costs, and the code · rewatch ▶ 2:03-2:47

4.1 4. $131,072 / 32,768 = 4$, the same factor as the episode's $64 \rightarrow 256$.

4.2 The KV cache (memory) grows with every token, and the model may still use the middle of a long text poorly. A longer window makes long inputs possible, not cheap or well used.

4.3 Interpolation only (3.59), no change (3.23), NTK (2.51), interpolation + 200 steps (1.58). From the episode's run; note that interpolation without training is worst even here.